Binary Search

leetcode

35.搜索插入位置

- 一个排序数组和一个目标值，在数组中找到目标值，并返回其索引。如果目标值不存在于数组中，返回它将会被按顺序插入的位置。时间复杂度为 `O(log n)` 的算法。
- 思路：二分查找寻找元素，存在则返回索引。不存在则返回下标。

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int n = nums.size();
        int left = 0,right = n-1;
        int ans = n;
        while( left<=right ){
            int mid = (left+right)/2;
            if(nums[mid]==target){
                ans = mid;
                break;
            }else if( nums[mid] < target ){ // 目标值太大，应该在右区间找,nums[mid]比target 小，则target肯定在mid的右边
                left = mid+1;
            }else{ // 目标值太小，应该在左区间找,nums[mid]比target大，说明target如果插入，至少可以插在mid这个位置
                right = mid-1;
                ans = mid;
            }
        }
        return ans;
    }
};
```

-

74.搜索二维矩阵

- 给一个矩阵（每行中的整数从左到右按非严格**递增顺序**排列，每行的第一个整数大于前一行的最后一个整数），在矩阵中判断target是否存在。

1	3	5	7
10	11	16	20
23	30	34	60

```
输入: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
输出: true
```

- 思路：将矩阵每一行拼接在上一行的末尾，得到一个升序数组，在该数组上二分找到目标元素。

```
class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        int m=matrix.size(),n=matrix[0].size();
        // 将二维矩阵拼接为一个一维矩阵，在一维矩阵上执行二分查找
        int left = 0,right = m*n-1;
        while( left <= right ){
            int  mid = (left+right)/2;
            int val = matrix[mid/n][mid%n];
            if( val==target ){
                return true;
            }else if( val<target ){ // 增大左区间
                left = mid + 1;
            }else{ // 缩小右区间
                right = mid - 1;
            }
        }
        return false;

    }
};
```

-

34.在排序数组中查找元素的第一个和最后一个位置

- 一个非递减顺序排列的整数数组 `nums`，和一个目标值 `target`。找出给定目标值在数组中的开始位置和结束位置。
- 思路：二分遍历查找target。
  - 寻找 leftIdx 即为在数组中寻找第一个大于等于 target 的下标。寻找 `>= target` 不仅能找到存在的 target 的左边界，还能在 target **不存在**时，告诉我们“target 应该插入在哪里”或者“第一个比 target 大的数在哪里”。
  - 寻找 rightIdx 即为在数组中寻找**第一个大于 target 元素**的下标（下标减1就是第n个target的元素的位置），然后将下标减一。

```
class Solution {
public:
    int binarySearch(vector<int>& nums, int target, bool lower){
        // 寻找第一个≥target的元素的下标（相当于把二分查找的等于和大于条件合并了）
        int left=0,right=nums.size()-1,ans = nums.size();
        while( left<=right ){
            int mid = (left+right)/2;
            // lower为true，nums[mid]>=target。lower为false，nums[mid] >
target
            if( nums[mid] > target || (nums[mid]>=target&&lower)){
                right = mid - 1;
                ans =  mid;
            }else{
```

```
                left = mid + 1;
            }
        }
        return ans;
    }

    vector<int> searchRange(vector<int>& nums, int target) {
        int leftIdx = binarySearch(nums,target,true); // 寻找第一个大于等于target的元素
        int rightIdx = binarySearch(nums,target,false) - 1; // 寻找第n个等于target元素（第一个大于taget的元素，然后下标-1)
        if( leftIdx <= rightIdx && rightIdx < nums.size() && nums[leftIdx]==target && nums[rightIdx]==target ){
            return vector<int>{leftIdx,rightIdx}; // 找到了
        }

        return vector<int>{-1,-1};

    }
};
```

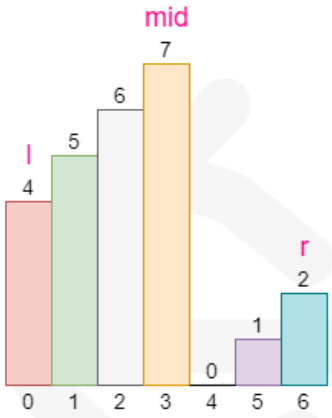
-

33.搜索旋转排序数组

- 整数数组 `nums` 按升序排列，数组中的值 **互不相同** 。然后对数组进行向左旋转，使数组变为 `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]`（下标 **从 0 开始** 计数）。现在给定一个旋转后的数组nums和一个整数target，找target的位置。

```
输入: nums = [4,5,6,7,0,1,2], target = 0
输出: 4
```

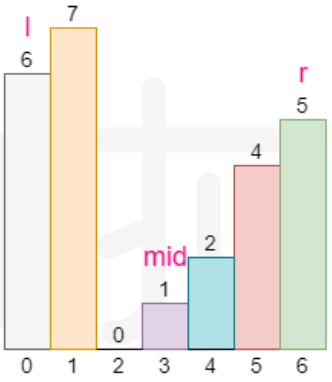
- 思路：对于任意一个index，**其左区间和右区间至少有一个是有序的**，那么就可以根据这个有序区间的最大值和最小值来**判断Target是否在该区间内**，由此就可以确定新的查找区间为有序半区还是无序半区。



[l, mid] 是有序数组

如果 target = 5，在 [l, mid - 1] 中寻找

如果 target = 2，在 [mid + 1, r] 中寻找



[mid + 1, r] 是有序数组

如果 target = 6，在 [l, mid - 1] 中寻找

如果 target = 4，在 [mid + 1, r] 中寻找

  - 如果 [l, mid - 1] 是有序数组，且 target 的大小满足 [nums[l],nums[mid])，则我们应该将搜索范围缩小至 [l, mid - 1]，否则在 [mid + 1, r] 中寻找。
  - 如果 [mid, r] 是有序数组，且 target 的大小满足 [nums[mid+1],nums[r]]，则我们应该将搜索范围缩小至 [mid + 1, r]，否则在 [l, mid - 1] 中寻找。

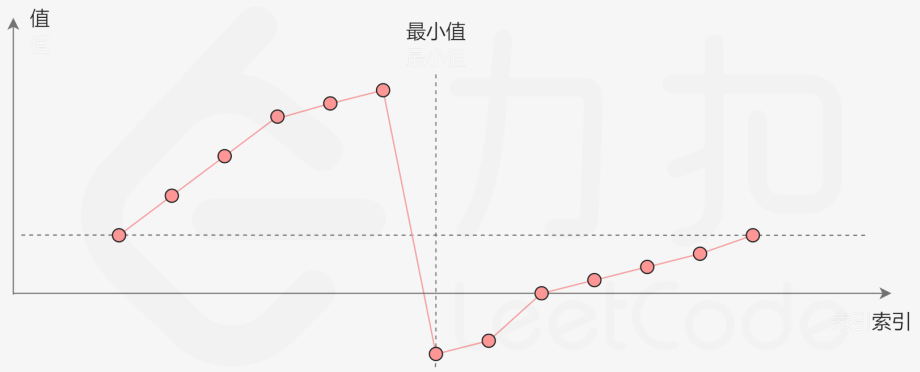
```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int n = nums.size();

        int left = 0,right = n - 1;
        while( left<=right ){
            int mid = (left+right)/2;
            if( nums[mid]==target ) return mid; // 找到了
            // 确定哪一边有序
            if( nums[0] <= nums[mid] ){ // 左边有序
                if( nums[0] <= target && target < nums[mid] ){ // target在左半区间
                    right = mid - 1;  // 在左半区间进行查找
                }else{ // target在右半无序区间
                    left = mid + 1;
                }
            }else{ // 右边有序
                if (nums[mid] < target && target <= nums[n - 1]) { // target在右半区间
                    left = mid + 1; // 在右半区间进行查找
                } else {
                    right = mid - 1;
                }
            }
        }
        return -1;
    }
};
```

-

153.寻找旋转排序数组中的最小值

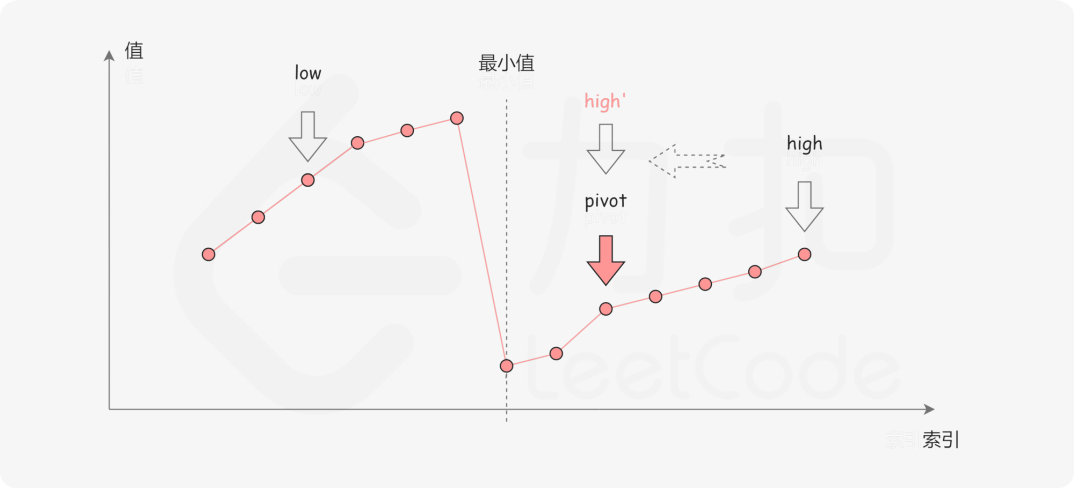
- 一个长度为 `n` 的数组，预先按照升序排列，经由 `1` 到 `n` 次 **旋转** 后，得到输入数组。数组 `[a[0], a[1], a[2], ..., a[n-1]]` **旋转一次** 的结果为数组 `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]` 。给你一个元素值 **互不相同** 的数组 `nums` ，它原来是一个升序排列的数组，**进行了多次旋转**。找出并返回数组中的 **最小元素** 。
- **思路**：**利用“中间值”与“右边界值”的大小关系，来判断最小值究竟在左半边还是右半边。**本质是寻找断崖点。



- ***nums*[*pivot*]<*nums*[*high*]**：说明 *nums*[*pivot*] 是**最小值**右侧的元素。

  `nums[pivot] < nums[high]`：

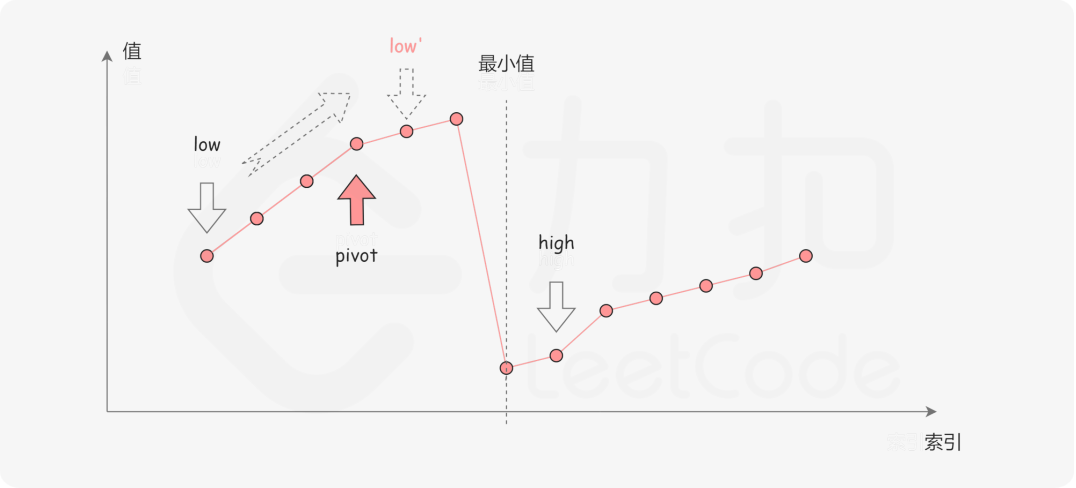
  - 说明从 `pivot` 到 `high` 是严格递增的（有序）。
  - 最小值不可能在 `pivot` 右边（因为右边的数都比 `pivot` 大）。
  - 但 `pivot` 自己有可能是最小值。
  - 操作：`high = pivot`（保留 `pivot`，缩小右边界）。



- ***nums*[*pivot*]>*nums*[*high*]**：说明 *nums*[*pivot*] 是**最小值**左侧的元素。

  `else`（`nums[pivot] > nums[high]`）：

  - 说明中间比右边大，发生了“断崖”。
  - 最小值一定在 `pivot` 的右边。
  - 且 `pivot` 自己肯定不是最小值（因为它比右边大）。
  - 操作：`low = pivot + 1`（排除 `pivot`，缩小左边界）。
  - *注：题目保证无重复元素，所以不存在相等的情况。*




```
class Solution {
public:
    int findMin(vector<int>& nums) {
        int n = nums.size();
```

```
        int low=0,high=n-1;
        while( low<high ){
            int pivot = (high+low)/2;
            if( nums[pivot] < nums[high] ){ // 从 pivot 到 high 是严格递增的（有序）
                high = pivot; // high可能是最小值
            }else{ // nums[pivot] > nums[high]，说明中间比右边大，发生了“断崖”
                low = pivot + 1;
            }
        }
        return nums[low];
    }
};
```

-